

Practical No:2

2. Write a program to implement Parallel Bubble Sort and Merge sort using OpenMP. Use existing algorithms and measure the performance of sequential and parallel algorithms.

Code:

```
#include <iostream>

#include <omp.h>

#include <cstdlib>

using namespace std;

// Print first 10 elements
void printArray(int a[], int n){
    int limit = (n > 10) ? 10 : n;
    for(int i = 0; i < limit; i++)
        cout << a[i] << " ";
    if(n > 10) cout << "...";
    cout << "\n";
}

// Sequential Bubble
void bubble(int a[], int n){
    for(int i=0;i<n-1;i++)
        for(int j=0;j<n-i-1;j++)
            if(a[j]>a[j+1]) swap(a[j],a[j+1]);
}

// Parallel Bubble
void pbubble(int a[], int n){
    for(int i=0;i<n;i++){
        #pragma omp parallel for
        for(int j=(i%2); j<n-1; j+=2){
            if(a[j] > a[j+1])
                swap(a[j], a[j+1]);
        }
    }
}

// Merge function
void merge(int a[], int l, int m, int r){
    int i=l,j=m+1,k=0,temp[r-l+1];
```

```

while(i<=m && j<=r)

    temp[k++] = (a[i]<a[j])?a[i++]:a[j++];
while(i<=m) temp[k++]=a[i++];
while(j<=r) temp[k++]=a[j++];
for(i=l,k=0;i<=r;i++,k++) a[i]=temp[k];
}

```

// Sequential Merge Sort

```

void msort(int a[], int l, int r){
    if(l<r){
        int m=(l+r)/2;
        msort(a,l,m);
        msort(a,m+1,r);
        merge(a,l,m,r);
    }
}

```

// Parallel Merge Sort

```

void pmsort(int a[], int l, int r){
    if(l<r){
        int m=(l+r)/2;
        if(r-l < 1000){ // threshold (avoid too many threads)
            msort(a,l,m);
            msort(a,m+1,r);
        } else {
            #pragma omp parallel sections
            {
                #pragma omp section
                pmsort(a,l,m);

                #pragma omp section
                pmsort(a,m+1,r);
            }
        }
        merge(a,l,m,r);
    }
}

```

```

int main(){
    int choice,n;

```

```
cout<<"1. Manual Input\n2. Auto Input (n=100000)\nEnter choice: ";
cin>>choice;
```

```
if(choice==1){
    cout<<"Enter n: ";
    cin>>n;
} else {
    n = 100000;
}
```

```
int *a = new int[n];
int *b = new int[n];
```

```
// Input
if(choice==1){
    cout<<"Enter elements:\n";
    for(int i=0;i<n;i++) cin>>a[i];
} else {
    for(int i=0;i<n;i++) a[i] = rand()%1000;
}
```

```
double t1, t2;
```

```
// Sequential Bubble
for(int i=0;i<n;i++) b[i]=a[i];
t1 = omp_get_wtime();
bubble(b,n);
t2 = omp_get_wtime();
cout<<"\nSeq Bubble: "<<t2-t1<<" sec\n";
printArray(b,n);
```

```
// Parallel Bubble
for(int i=0;i<n;i++) b[i]=a[i];
t1 = omp_get_wtime();
pbubble(b,n);
t2 = omp_get_wtime();
cout<<"\nPar Bubble: "<<t2-t1<<" sec\n";
```

```
printArray(b,n);

// Sequential Merge
for(int i=0;i<n;i++) b[i]=a[i];
t1 = omp_get_wtime();
msort(b,0,n-1);
t2 = omp_get_wtime();
cout<<"\nSeq Merge: "<<t2-t1<<" sec\n";
printArray(b,n);

// Parallel Merge
for(int i=0;i<n;i++) b[i]=a[i];
t1 = omp_get_wtime();
pmsort(b,0,n-1);
t2 = omp_get_wtime();
cout<<"\nPar Merge: "<<t2-t1<<" sec\n";
printArray(b,n);

delete[] a;
delete[] b;
return 0;
}
```

```

PS C:\Users\Rutuja Varunkar\Downloads\hpc> g++ -fopenmp P2.cpp -o P2
PS C:\Users\Rutuja Varunkar\Downloads\hpc> ./P2
1. Manual Input
2. Auto Input (n=100000)
Enter choice: 1
Enter n: 7
Enter elements:
32
89
76
3
0
87
1

Seq Bubble: 0 sec
0 1 3 32 76 87 89

Par Bubble: 0.00399995 sec
0 1 3 32 76 87 89

Seq Merge: 0 sec
0 1 3 32 76 87 89

Par Merge: 0 sec
0 1 3 32 76 87 89

```

```

PS C:\Users\Rutuja Varunkar\Downloads\hpc> g++ -fopenmp P2.cpp -o P2
PS C:\Users\Rutuja Varunkar\Downloads\hpc> ./P2
1. Manual Input
2. Auto Input (n=100000)
Enter choice: 2

Seq Bubble: 27.357 sec
0 0 0 0 0 0 0 0 0 ...

Par Bubble: 8.777 sec
0 0 0 0 0 0 0 0 0 ...

Seq Merge: 0.00800014 sec
0 0 0 0 0 0 0 0 0 ...

Par Merge: 0.023 sec
0 0 0 0 0 0 0 0 0 ...
PS C:\Users\Rutuja Varunkar\Downloads\hpc> █

```